Agentic Travel Planning: Where AI Helped, and Where It Did Not

Stefan Schneider
stefanbschneider@outlook.com

Fabian Missbrenner
mail@fabimi.net

Abstract

Many agentic tasks begin with an open-ended goal but are useful only when hard constraints are met. Multi-day travel planning is a popular example for such a task. We describe our entry to the "2nd Travel Planning Challenge with Agentic AI" at the IJCAI 2026 conference[1]. Curiously, we found that the best-performing system was a deterministic, environment-grounded planner that completely ignores the available large language model (LLM). Instead, it uses classical optimization and acceptance gates that can only emit itineraries confirmed valid against the environment rules and APIs.

Conversely, AI agents were most valuable in building and improving that system: We successfully used the competition as a low-risk environment to run a near-continuous data-driven multi-agent AI development process, generating hundreds of commits over eight weeks. The agents proposed experiments, implemented changes, ran evaluations, and reviewed pull requests, while humans made every decision that mattered: what to prioritize and how to split the work, how to measure progress and provide a fast, meaningful feedback loop for the agents, and what to finally submit.

1 Introduction

Many real-world tasks involve a loosely stated human goal that needs to be turned into an output that is not merely plausible but correct. For example, multi-day travel planning combines fuzzy human requirements that must be turned into exact constraints, fulfilled by the constructed itinerary. The IJCAI 2026 Travel Planning Challenge[1] extends the ChinaTravel benchmark of Shao et al. [Shao *et al.*, 2024]: for a given human travel intent, an agent must produce a preference-aligned itinerary between a selection of Chinese cities. A request might say:

> "Plan three days in Suzhou, visit the Humble Administrator's Garden between 9 and 11, stay near the old town, avoid taxis, and spend at most CNY 3,000."

[1]Website: https://chinatravel-competition.github.io/IJCAI2026/, Benchmark: https://github.com/LAMDA-NeSy/ChinaTravel (URLs accessed on August 9, 2026)

The output is structured data, not free text. Hotels, attractions, and restaurants must exist in the provided sandbox environment; opening hours, transportation, and costs must be consistent; logical requirements must hold; and commonsense rules such as meal counts must pass. The evaluator measures environmental validity (EPR), logical satisfaction (C-LPR and FPR), and preferences. Hard constraint satisfaction heavily dominates the overall score – preference terms only count if all hard requirements are fulfilled (else set to zero).

The competition has two phases. In the offline first phase, released verifier fields and a generous time budget allowed us to satisfy almost all constraints and then polish preferences. In the harder phase 2, which we focus on in this report, only query text and environment tools are exposed to the agent. For each query, the agent must infer what constraints to satisfy and how – within 5 hours total for 100 unknown queries.

Our system follows a propose–verify–repair pattern related to LLM-Modulo systems [Kambhampati *et al.*, 2024], but assigns roles asymmetrically: models may interpret language or propose changes; environment APIs establish facts; and deterministic optimization and checks decide what may be emitted.

2 System Design

2.1 Pipeline and trust boundary

Figure 1 follows the example from language to a simplified itinerary. Oracle verifier fields are removed before execution (only available in the public data, not the held-out test set). The only inputs are visible query text and allowed environment tools. Every point of interest (POI), hotel, restaurant, price, opening hour, and transportation record is read from `WorldEnv`; nothing is generated as a plausible substitute.

2.2 Parsing and normalization

The parser extracts dates, cities, budgets, excluded transportation, named POIs, timed visits, hotel properties, room counts, cuisines, and attraction categories. It also handles alternatives ("visit A *or* B") by creating separate solver variants rather than an impossible conjunction. Grounding and normalization resolves names and concepts against each city's catalog. Deterministic extraction is fast and testable but not synonymous with understanding, so we added phrasing-robust fallbacks for the largest measured recall gaps and an

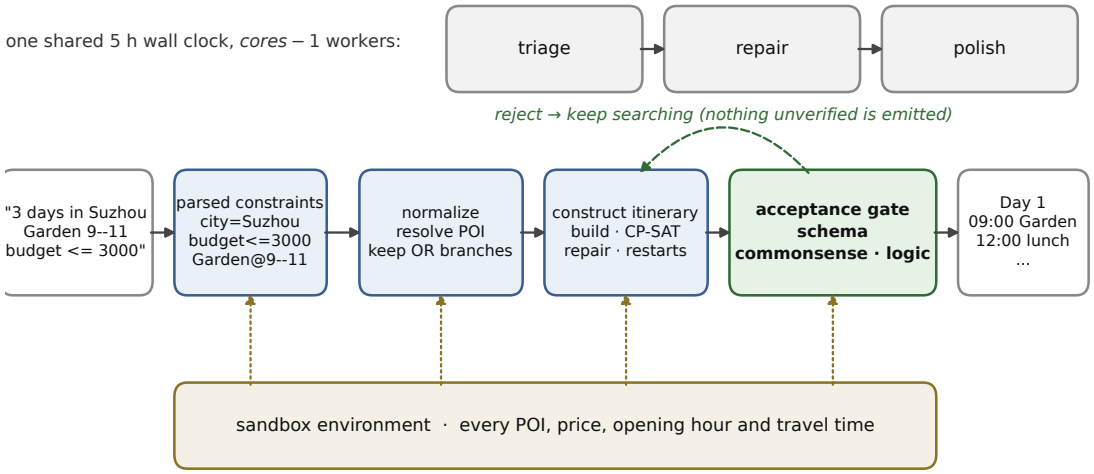


Figure 1: The per-query pipeline and whole-run scheduler, illustrated with the opening example. Optional model-based extraction may fill missing constraints; grounding, construction, repair, and acceptance remain deterministic.

optional LLM[2] extractor that may fill a missing family but never overwrite a deterministic value.

Precision is equally important. Our audit found that in 23–50% of free-prose queries, depending on the set, the parser initially invented a required POI name that did not exist in the target city. Such a query has no feasible solution at all, so it can never earn a full-plan point no matter how long the solver runs. Therefore, a catalog check in the final parser removes such provably unsatisfiable required names and category values, after first trying to salvage a real POI name from the captured text. It reduced this measured failure class to zero on every set we hold.

Further investigation illustrates how narrow these parsing gaps are. Until a recent fix to our final agent, intercity transport was the largest remaining unextracted family on the free-prose dataset: the extractor recognized "take the train to the destination" but not "we have chosen to fly" or "by high-speed train (G)", missing 33 of 85 such requirements on one released set and 27 of 32 on another. Adding verb-form and qualified-noun patterns, guarded so that a station name or an undecided "fly or take the train" never produces a requirement, removed most of them while leaving precision and the familiar register unchanged.

2.3 Construction, repair, and densification

The deterministic planner constructs a complete base itinerary from grounded candidate pools. It then applies bounded CP-SAT [Perron and Didier, 2024], domain-specific repairs, and seeded restarts. Repairs include placing named activities inside requested windows, inserting absent required POIs, closing transportation gaps, and rebuilding intercity legs. The same acceptance checker evaluates every candidate: query constraints, schema, and commonsense rules must all pass. Accepted candidates are ranked by preference value and cost.

[2]The organizers provide a Qwen3.6-27B model for evaluation.

Metric	Familiar 100	Held-out v1	Best held-out
Overall	64.17	33.25	36.41
EPR micro	99.67	99.48	99.64
EPR macro	98.00	95.00	95.00
C-LPR	90.08	55.20	67.77
FPR	44.00	0.00	0.00

Table 1: Local familiar-register baseline, our official v1 result, and the leading team's v1 result on the same held-out set.

Packing and densification add meals and activities only while the checker keeps passing. A minimal schema-valid itinerary is written before solving so an interrupted run still has output — availability, not success, since an empty itinerary scores zero.

2.4 Scheduling the five-hour run

The whole-split scheduler uses *triage*, *repair*, and *polish* passes in parallel across queries: triage reaches every query cheaply, repair revisits the nearest failures with larger budgets, and polish spends what is left only after hard requirements pass. A keep-best rule makes updates monotonic, and deadline checks prevent starting work that cannot finish.

3 Evaluation

3.1 Official feedback

In the final, second phase of the competition, the organizers permit two submitted harness versions and retain the better score. We submitted v1 early to obtain the only available held-out feedback before choosing v2. Table 1 compares that result with the current run on 100 familiar released queries. The columns are diagnostic signals, not identically distributed test sets.

Grounding generalized: held-out micro EPR remained 99.48, within 0.16 of the best entry. Constraint understanding

did not: C-LPR fell to 55.20, no query passed every requirement, and all preference terms were therefore zero.

The full field is instructive. Across the five submitted systems, four score a full-plan rate of exactly zero and the fifth scores one point, and every preference term is zero for every team. The held-out queries are evidently dense enough that no entrant reliably satisfies all clauses of a single query, so a zero there is a property of the benchmark at this difficulty, not a defect unique to one system. It also means 55% of the metric weight currently pays nobody, and the ranking is decided almost entirely by the per-clause rate at weight 0.25: the entire 3.16-point gap between our v1 and the leading entry is 12.6 clause points. Because the full-plan term is multiplicative in the per-clause rate, it stays near zero until roughly 0.8 and then rises steeply; the first system past that threshold collects the dormant weight in one step.

3.2 What the second version changes

Since the v1 build, we repaired the phrasing brittleness described below, removed the unsatisfiable-name failure class, and closed the largest remaining recall gaps. None of that is visible on the familiar register, where the extractor was already complete, so we measured it where it applies: 150 differently phrased released queries, both builds run as a paired experiment at identical budget on one host, scored against an English-literal oracle so the full-plan rate has a real denominator.[3] Figure 2 shows the result: the clause rate rises from 75.6 to 92.3 and the full-plan rate from 10.7 to 65.3. Checked clause by clause, 225 of 1264 clauses move from failing to passing against 13 lost, and the number of queries satisfying every requirement rises from 16 to 98 of 150. On the familiar register, where most of this cannot help, a paired run of the full five-hour pipeline confirms no regression and a modest gain: 64.69 to 66.81 overall, with the full-plan rate moving from 45 to 49 of 100 queries.

The largest single fix illustrates a measurement lesson. Our coverage audits classified the hotel room count as "guaranteed by trip metadata", because the planner derives it from party size and beds per room — correct only while the traveler says nothing. Asking a different question, *for each query that fails exactly one requirement, which requirement is it?*, showed that 28 of 150 free-prose queries failed *only* the room count: one field away from satisfying everything. Reading the stated count instead of deriving it moved the full-plan rate on that set from 48% to 65%. No coverage table could have found this; a near-miss analysis found it in minutes.

3.3 Query phrasing

The familiar released set was presented as in distribution, so prioritizing it was a reasonable choice. Its queries use a regular numbered template, whereas other released sets express the same requirements in varied prose: offline audits found no missed clauses on the familiar register but 12–29% on the differently phrased ones. We had optimized against the one register we were already perfect on.

[3]Those released sets state their query text in English but their verifier literals in Chinese, which caps the achievable full-plan rate; we translate the literals with an audited one-to-one map for measurement only, and the agent never reads them.

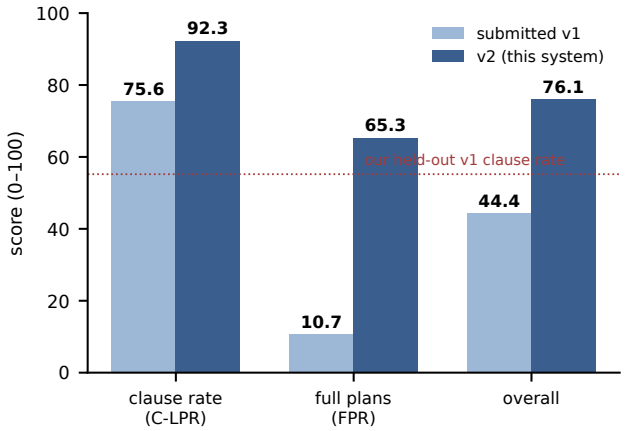


Figure 2: Submitted v1 versus the second version on 150 unfamiliar-phrasing queries, paired at equal budget. The dotted line is our v1 clause rate on the organizers' held-out set, a different query set, shown for scale.

3.4 Planning baselines

We ran two small, matched comparisons. An autonomous tool-using Qwen agent scored 1.88 and 2.64 on eight familiar and eight free-prose queries, versus 67.71 and 48.49 for our solver; the upstream RuleNeSy baseline [Shao *et al.*, 2024] scored about 9.5 on four of each. Neither produced an itinerary passing every commonsense rule. These samples are too small to rank systems, but the failure modes support retaining deterministic acceptance and optimization.

4 Where AI Helped — and Where It Did Not

4.1 The competition as a safe vibe-coding experiment

We used this challenge deliberately as a low-risk environment in which to push agent-driven development much harder than we would on a production system: an objective automatic scorer, nothing deployed, and wrong changes caught by a test rather than an incident. It let us ask an otherwise expensive question — how far can coding agents carry a real system given fast, honest feedback and no human in the inner loop?

Very far, as it turned out, given a great deal of scaffolding. We ran multiple paid agent subscriptions across several machines, with one or more agents working almost continuously for eight weeks. The result is 798 commits and 100 merged pull requests over 109 opened ones, a suite of 1339 tests, and 47 committed measurement screens alongside 63 recorded evaluation rows. Humans still made every decision that mattered: what to measure, what a measurement was allowed to prove, and what to submit.

Three practices did the actual work.

Split the roles by cost and by kind of thinking. Expensive reasoning models were used for analysis, diagnosis, and planning: reading measurements, proposing the next experiment, ranking work by expected effect. Faster, cheaper agents then implemented a specified change with tests, and ran evaluations. The planning agent maintained a ranked

work list; implementation and evaluation proceeded in parallel branches against it, which also avoided the common failure where an implementing agent silently redefines the task it was given.

Make feedback fast, automatic, and cheap to trust. We invested more in the evaluation harness than in any single planner feature: a paired A/B screen that decides in one or two hours whether a change is worth a long run, a whole-run mode reproducing the submission's exact shape, deterministic seeds, and result ledgers recording every condition needed to decide whether two numbers may be compared. Agents could then evaluate their own work overnight and report a verdict against a criterion written before the run.

Write the invariants down and let them be enforced by tests. Grounding, no oracle leakage, acceptance-before-emission, and flags-off-is-the-floor are project rules pinned by tests, so an agent that would violate one fails rather than persuades. Cold-start verification caught a start-up race that would have shipped empty itineraries for a whole split while still exiting successfully.

Two limits are worth stating. The volume of agent output is not itself a result: much of it was superseded, and the ratio of committed work to accepted improvement was low. And agents were reliably over-optimistic about their own changes, which is why promotion required an independent measurement.

4.2 Inside the system

Natural-language interpretation is a more nuanced case. LLM-first extraction cut missing free-prose constraint families by 40–65%, but initially converted many misses into values that did not match the environment vocabulary. We therefore connected model output to the same city-specific entity and category catalogs used by the solver. Grounded, fill-only extraction improved one small familiar A/B but was flat on the matched free-prose subset, and still matched fewer clauses than the contemporaneous deterministic parser. The current default consequently makes zero LLM calls, while retaining the grounded extractor as an optional path. This is not a rejection of LLM parsing; it separates recognizing a requirement from resolving its values against the actual database.

The strongest argument for that path is what our deterministic extractor cannot reach. Its largest remaining gap on unfamiliar phrasing is no longer lexical: the queries say "we are more interested in history and culture" or simply "going on a date, please recommend an itinerary". Each implies a database category that no pattern names, and the benchmark's own labels for such phrasings are inconsistent, so hand-written rules would be guessing. Inferring intent is what a language model is for, through the existing tools or an MCP-style interface; its answer only has to be resolved against the closed vocabulary first.

We also tried the obvious hybrid: letting a model repair itineraries the checker rejected, with every proposed edit re-verified before it could be kept. On ten deterministic failures the loop iterated, proposed valid-looking edits, and flipped none of them to passing. It is off by default and reported here as a negative result rather than omitted.

End-to-end itinerary generation was the clearer boundary. Even with environment tools, autonomous Qwen planning accumulated small inconsistencies, and one violation invalidated the whole itinerary. Dates, opening windows, travel times, budgets, meals, and disjunctions remain a constrained optimization problem, where deterministic solving provides reproducibility, explicit acceptance, and efficient targeted repair.

5 Conclusion

The Travel Planning Challenge rewarded a hybrid division of labor. AI agents accelerated the construction and analysis of the system, and model-based extraction recognized language missed by deterministic patterns. Environment tools supplied facts; deterministic search assembled them; and the acceptance gate ensured that only verified itineraries could be emitted. The official v1 result sharpened that boundary: grounding transferred to hidden data, while constraint extraction tuned to one advertised in-distribution phrasing register did not — and repairing that is what the second version measurably contains.

The transferable lesson is methodological. Agentic coding works particularly well in a small, isolated project once objective and fast feedback exists — and this challenge was chosen partly to test that at a scale of continuous multi-agent operation we would not risk on a production system. What made it work was not the agents but the harness around them: written invariants enforced by tests, a measurement contract that says when two numbers may be compared, and a rule that only an independent run may promote a change. Models are valuable at the fuzzy boundary between language and symbols, provided their proposals are grounded. Hard itinerary construction remains optimization.

A production system would replace the sandbox with live map, hotel, rail, flight, and booking APIs, and would infer user-specific objectives rather than inherit fixed challenge preferences: some travelers want four attractions per day, others value unplanned time. Live availability would require replanning, but the same architecture holds. Let AI interpret intent and help build the system; let APIs establish reality; let optimization construct the itinerary; and let deterministic verification decide what is safe to emit.

Acknowledgments

We thank the challenge organizers for the benchmark, the agent environment tooling, and their clarifications and answers to our questions.

References

[Kambhampati *et al.*, 2024] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. LLMs can't plan, but can help planning in LLM-Modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[Perron and Didier, 2024] Laurent Perron and Frédéric Didier. CP-SAT. https://developers.google.com/optimization/cp/cp_solver, 2024. Google OR-Tools.

[Shao *et al.*, 2024] Jie-Jing Shao, Bo-Wen Zhang, Xiao-Wen Yang, Bai-Zhi Chen, Si-Yu Han, Jing-Hao Pang, Wen-Da Wei, Guohao Cai, Zhenhua Dong, Lan-Zhe Guo, and Yu-Feng Li. ChinaTravel: An open-ended benchmark for language agents in Chinese travel planning. *arXiv preprint arXiv:2412.13682*, 2024.